1. 開発目的：大規模量子回路を実行したい

分散量子回路シミュレータの開発

Deferred Hierarchical Distributed State-Vector Simulator（DH-SVS、仮称）

容量の壁：dense 状態ベクトル

$$S(n) = 16 \times 2^n \text{ bytes}$$

`complex128`、状態 1 コピーの容量

量子ビット数	状態容量
32	64 GiB
40	16 TiB
50	16 PiB

作業 Block、通信バッファ、Adjoint 状態、Checkpoint、ログなどは別途必要。

目標：容量と実行時間を両立する

- 単一ノード RAM を超える状態を扱う。
- RAM・SSD/HDD・MPI を統合する。
- 全状態走査と転送の繰り返しを減らす。

評価対象は End-to-End 時間

RAM 帯域 / ディスク I/O / MPI 通信 / 同期待ち / 振幅演算 / 計画の費用

保存できる容量と**時間内に計算できる規模**を分けて評価する。

本資料は実装前の設計説明。高速化効果・研究上の新規性は今後検証する。

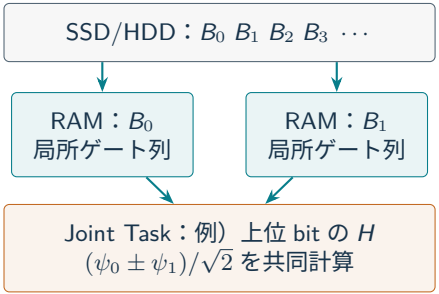
2. 基本戦略：状態ベクトルを Block に分割する

振幅区間を管理・実行の単位にする

$$|\Psi\rangle = \sum_{h=0}^{2^{n-b}-1} |h\rangle \otimes |\psi_h\rangle\,, \quad |\psi_h\rangle \in \mathbb{C}^{2^b}$$

- b：Block 内のインデックスビット数。
- Block-local 演算は、RAM 滞在中にまとめて実行。
- Block 間の振幅混合は、必要な Block 群の **Joint Task** で処理。

Block は独立した量子部分系ではない。未正規化の振幅と Block 間の相対位相を保つ。



Global DAG で入力版・依存順序を保証

Block を読み込むたびに**複数の演算を進める**ことで、再読み込み・書き戻しを減らす。

3. 全体行列の形が、読み出す振幅ペアを決める

2 qubit、基底 $|q_1 q_0\rangle$、$b = 1$（2 振幅/Block）。v は Frame・相対位相を反映した論理振幅。

$$v = (v_0, v_1 \mid v_2, v_3)^{\mathsf{T}}, \qquad B_0 = (v_0, v_1)^{\mathsf{T}}, \quad B_1 = (v_2, v_3)^{\mathsf{T}}$$

q_0 への U：$I \otimes U$

	00	01	10	11
00	u_{00}	u_{01}	0	0
01	u_{10}	u_{11}	0	0
10	0	0	u_{00}	u_{01}
11	0	0	u_{10}	u_{11}

同じ Block 内のペア
(v_0, v_1)、(v_2, v_3)

$B_0' = UB_0, \quad B_1' = UB_1$
各 Block を RAM へ読み、Block 内で更新できる。

q_1 への U：$U \otimes I$

	00	01	10	11
00	u_{00}	0	u_{01}	0
01	0	u_{00}	0	u_{01}
10	u_{10}	0	u_{11}	0
11	0	u_{10}	0	u_{11}

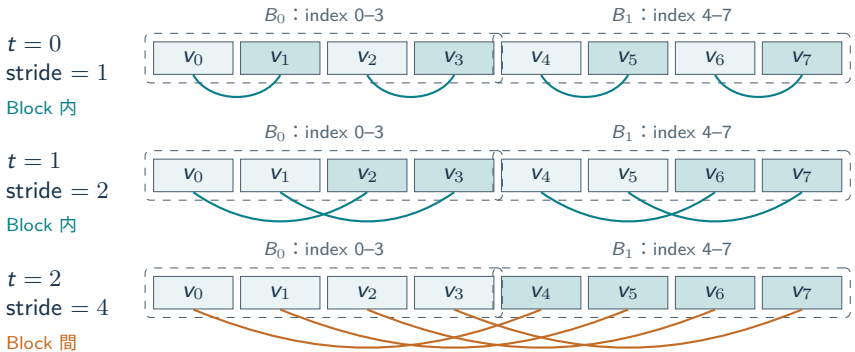
Block を跨ぐ同じ offset のペア
(v_0, v_2)、(v_1, v_3)

$B_0' = u_{00} B_0 + u_{01} B_1$
$B_1' = u_{10} B_0 + u_{11} B_1$

同じ 2×2 ゲートでも、**作用する bit の位置で必要な Block が変わる**。全体行列は生成しない。

4. ストライドアクセス：どこで Block 境界を越えるか

3 qubit、q_0 が最下位 bit。1 Block = 4 振幅（$b = 2$）。線は U で同時更新するペア。



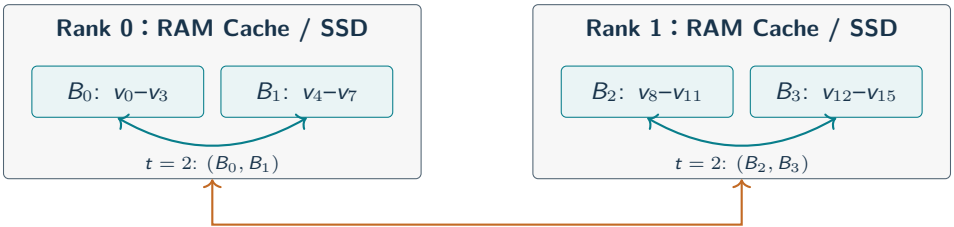
i の bit t が 0 なら、$j = i + 2^t$。$\begin{pmatrix} v_i' \\ v_j' \end{pmatrix} = U \begin{pmatrix} v_i \\ v_j \end{pmatrix}$：両入力を保持してから両出力を書く。

$t < b$ なら同じ Block、$t \geq b$ なら別 Block の同じ offset をアクセスする。

5. Block ペアの取得：Rank 内 I/O と MPI を区別する

連続 Block 配置の例：4 qubit、$b = 2$、2 rank。bit t は現在の物理 Layout で解決した位置。

$$i = h\,2^b + \ell, \quad 0 \le \ell < 2^b, \qquad t \ge b: \quad h' = h \,\mathrm{XOR}\, 2^{t-b}$$



同じ offset を共同更新

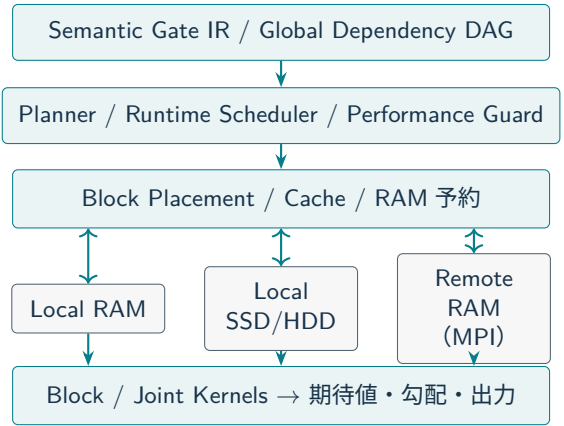
$(B_h[\ell], B_{h'}[\ell])$ へ U を適用。
Rank 内でも Cache miss なら SSD 読出し。
対角 U なら Block 間の振幅混合は不要。

Joint Task の実行手順

入力版確認 → RAM 予約 → 2 Block 取得。
Frame・相対位相整合 → 更新 → 出力版確定。
次の局所ゲートも RAM 滞在中に処理する。

Block 境界を越えることと Rank 境界を越えることは別。取得先は Layout・所有者から解決する。

6. MPI + RAM + SSD/HDD を同じ配置モデルで扱う



必要な Block の正しい版を取得

1. Local RAM を再利用。
2. Local SSD/HDD から読む。
3. MPI で Remote RAM から受信。
4. 同じ base 版＋ Log から再生。

取得経路の選択条件

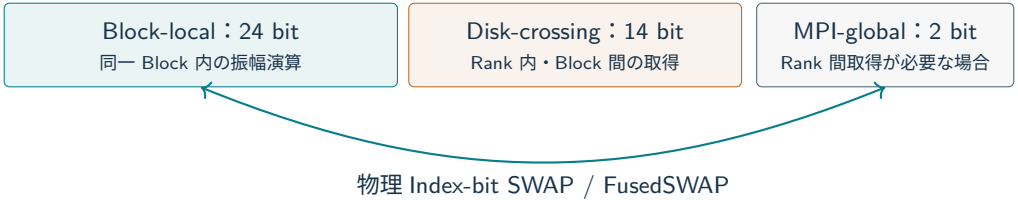
Block 版・依存 frontier・Frame・Layout が整合する候補だけを、移動／再生の費用で比較する。

Checkpoint は振幅だけでなく、版・Log・Frame・Layout の整合した情報を保存する。

Out-of-core と MPI 分散を**「必要な振幅を計算場所へ集める」**問題として統合する。

7. Qubit Layout：再配置の費用を後続ゲートで回収する

例：40 qubit、4 MPI rank（$g = 2$）、Block 内 $b = 24$。領域は現在の物理配置で決まる。



2 種類の SWAP を区別する

- **論理 SWAP**：入力回路の作用。後続 wire 対応の更新で表せる場合がある。
- **物理再配置**：論理状態を保ち、振幅配列と Layout を一緒に変更する。

先読み窓の総費用で選択する

$$T_{\mathrm{move}} + T_{\mathrm{after}}(W) + T_{\mathrm{align}} < T_{\mathrm{direct}}(W)$$

W：同じ先読み窓。移動＋窓内全ゲート＋整合の費用を、現 Layout での窓全体と比較。必要な復元費用も加える。

FusedSWAP は複数の配置変更をまとめる候補。効果は転送量と実時間で測る。

MPI 削減量だけでなく SSD/HDD の増分 I/O も含めて、局所化の損益を判断する。

8. 遅延 Log と Clifford Frame：必要時にまとめて具体化

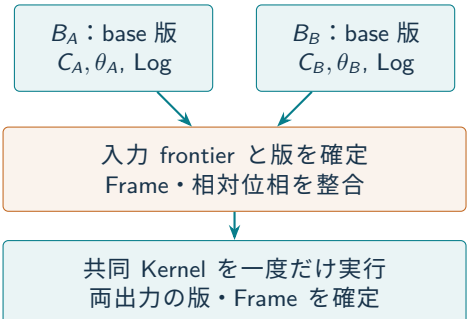
保存振幅と論理状態を分ける

$$|\psi_h\rangle = e^{i\theta_h} C_h |\phi_h\rangle$$

整合した論理 cut での表現。ϕ_h：保存振幅、C_h：Block 内 Clifford、θ_h：相対位相。

- Per-Block Log：局所演算の履歴。
- Global DAG：Block 間依存と、一意な Joint Task を保持。
- Frame Group：共通 Frame を共有し、メタデータ増大を抑える。

frontier ＝論理的にどこまで演算を適用したか。保存振幅の base 版とは別に管理する。



例：上位 bit の H は Block 振幅を混合する

遅延実行で走査をまとめる。**全振幅の具体化コストや状態の情報量は消えない。**

9. Pauli Frame で保持できる作用と、H の扱い

Block 内 Pauli Frame：$|\psi_h\rangle = e^{i\theta_h} P_h |\phi_h\rangle$、$P_h = X^{x_h} Z^{z_h}$。積の位相・符号は θ_h へ保持。

局所 X/Z：振幅を読まず Frame へ吸収

保存 Block ϕ_h：変更しない

Frame P_h
位相 θ_h
→ Q →
Frame P'_h
位相 θ'_h

$QP_h = e^{i\delta_h} P'_h$、$\theta'_h = \theta_h + \delta_h$。
例：$P_h = I$ へ X を適用 $\Rightarrow P'_h = X$。
Pauli 積は再び Pauli。Block 間の相対位相を捨てない。

H そのもの：Pauli Frame への吸収は不可

$\phi = |0\rangle$
$P = I$
→ H →
$H|0\rangle = |+\rangle$
2 振幅の重ね合わせ

位相を除くと I, Z は $|0\rangle$、X, Y は $|1\rangle$。
保存 $|0\rangle$ を固定したまま、Pauli ＋位相だけで $|+\rangle$ は表せない。
H は Clifford なので、Block 内 Clifford Frame なら保持できる。

既存の Pauli Frame は H 越しに伝播できる：$HX = ZH$

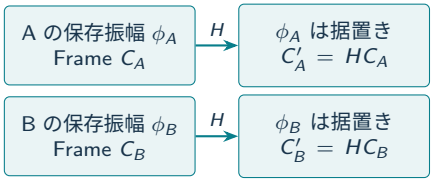
論理状態 $X|\phi\rangle$
保存状態 ϕ、Frame $P = X$
→
H を適用：$HX|\phi\rangle = ZH|\phi\rangle$
Frame は $X \to Z$ へ更新
→
残る H は ϕ へ作用させる
または Clifford Frame へ保持

Pauli の伝播と、H そのものの遅延は別。$HXH = Z$、$HZH = X$、$HYH = -Y$ の符号も保持する。

10. 同じ H でも：Block 内は Frame、Block 間は共同演算

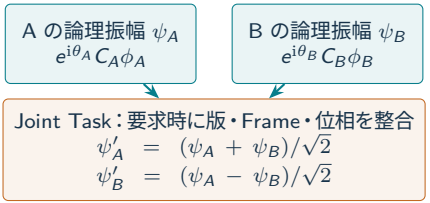
2 qubit、基底 $|q_1 q_0\rangle$、$b = 1$。q_0 は Block 内、q_1 は Block 番号の bit。以下は整合した論理 cut での表現。

$H(q_0)$：Block 内 Clifford Frame で追跡



A/B 間の振幅混合はない

$H(q_1)$：Per-Block Frame だけでは不可



両入力を保持し、両出力版を一度だけ公開

局所 H は C_h の更新で遅延できる。
Pauli Frame から Clifford Frame へ表現を広げる。

Global DAG へ登録して遅延。必要時に Block ペアを取得し、和と差を共同計算する。

上位 Z：B の位相 $+\pi$
振幅混合なし

上位 X：論理参照 A/B を交換
版・Frame 等も一緒に対応

上位 H：A/B の和と差
位相・置換だけでは表せない

局所 H は Clifford Frame へ、上位 H は Joint DAG へ。振幅の和と差が必要な時点で具体化する。

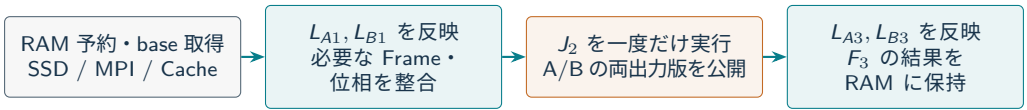
11. 遅延 Log から振幅へ反映するタイミング

登録時は Log・Frame・DAG を更新。下図は、後続 Task が Block A/B の frontier F_3 の振幅を必要とする例。



矢印は依存関係。J_2 の登録だけでは振幅を更新しない。

要求から必要な依存を遡り、合法な順序で materialize する



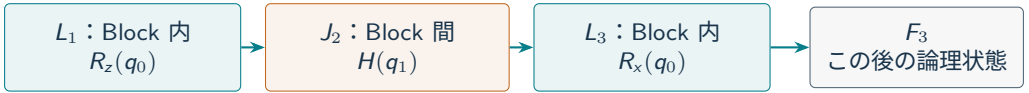
必要になったとき
次の Joint 入力、出力・観測・測定の要求。
要求 frontier に必要な Block と Log を解決する。

先に実行した方が得なとき
RAM 滞在中の再利用、再生費用・ログ量の閾値。
背景実行は I/O・MPI 競合も含めて判断する。

必要な依存を満たす時点でまとめて反映する。Frame-aware 観測は振幅の具体化を省ける場合がある。

12. F_3 はどの操作の後の状態か

F_3 はゲート名ではなく、Log・Joint Task のどこまでを反映したかを示す論理 frontier。



F_3 を要求する操作の例

- 次のゲート：例えば次の $H(q_1)$ に必要な入力振幅。
- 振幅出力：この回路 prefix 後の振幅を読み出す。
- 観測・測定：この時点の状態から値・結果を求める。

要求に応じて必要な範囲だけ進める

- F_3 より前の依存を遡る。
- $L_1 \to J_2 \to L_3$ の順に版をそろえて実行。
- 要求 Block の F_3 版を RAM へ保持・提供する。

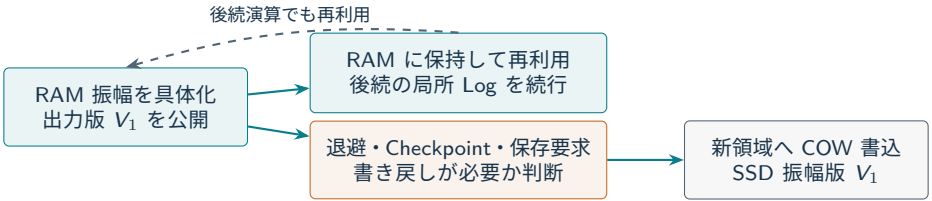
観測時の補足

Frame-aware な Pauli 期待値などは、必要な Block 版・Log・Frame・相対位相を解決して直接計算できる場合があり、振幅の全面的な具体化を省ける。

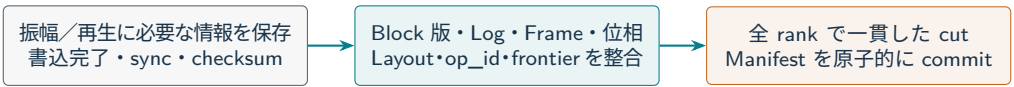
F_3 は「3 番目のゲート」ではなく、L_1・J_2・L_3 **を反映した状態の版**を指す。

13. RAM の更新結果を、いつ SSD へ書き戻すか

振幅を RAM で更新する materialization、後続 Task への版の公開、SSD への永続化を別の段階として扱う。



Checkpoint：耐久性を保証してから Manifest を公開

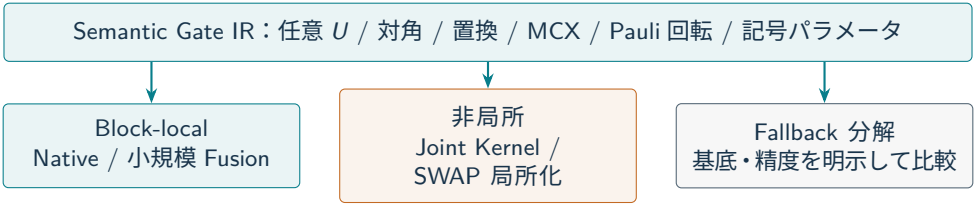


Cache 退避時
同版 Replica が有効なら追加書込は不要。
dirty 版は保存、または正しい base ＋ Log で安全に再演算できる場合に限り破棄する。

未具体化 Log を含む Checkpoint も可能
base・Frame・位相・Log・未実行 DAG などを保存し、全状態を再生できることが条件。
Replica も耐久条件を満たす必要がある。

毎ゲート書き戻す必要はない。RAM の再利用と再生可能性を見て、必要な永続化をまとめる。

14. Native-first：ゲートの意味と構造から実行方法を選ぶ



専用 Kernel を優先候補にする

- 1Q/2Q 小密行列、対角・置換。
- MCX：X / CNOT / CCX / CCCX 以上を条件付き振幅置換として統一。
- 任意 U を検証して受付。実行幅・行列容量の上限は実測で決める。

パラメータ変更で計画を再利用

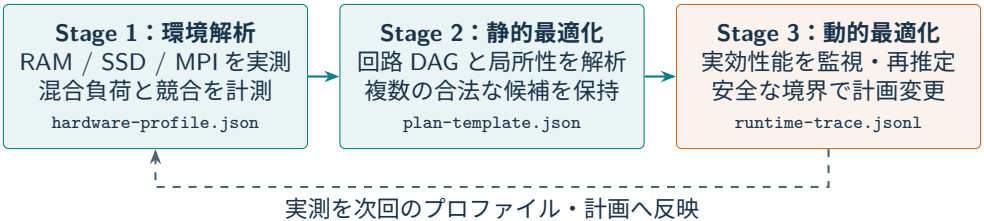
$$R_z(\alpha)R_z(\beta) = R_z(\alpha+\beta)$$

記号式を融合 → 値を bind → 値依存候補を guard 付きで特殊化。

prefix 状態の再利用は、依存パラメータ・入力状態・frontier・Frame・Layout が一致する場合のみ。

Native・Fusion・Joint・SWAP・分解を、**精度とデータ移動を含む総費用**で比較する。

15. 3 段階の最適化：計測→計画→実行時適応



Performance Guard：劣化を検知

- 同じ I/O サイズ・キュー深度・通信形・並行負荷の想定値と比較。
- 移動平均・継続時間・ヒステリシスで一時的な揺らぎを除く。

効果と切替費用を比較して変更

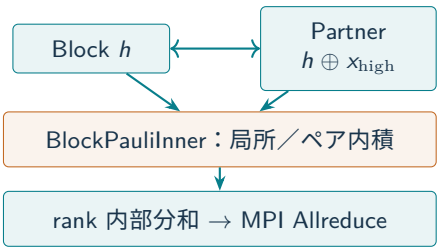
- 先読み深さ・並列度・取得元は Task 境界で調整。
- Layout 等は版・DAG・Frame が整合する **safe frontier** で確定。

残りの実行時間の短縮が、**監視・再計画・移行の追加費用**を上回るときに切り替える。

16. 期待値・Backpropagation も Block 単位で実行する

Pauli 期待値：部分和＋ MPI 集約

$$E = \langle \Psi | O | \Psi \rangle, \quad O = \sum_{\ell} c_\ell P_\ell$$



x_{high}：Pauli の X/Y が反転する上位 bit の mask。全状態を 1 ノードに Gather せず計算する。

Adjoint：2 状態を逆方向に伝播

$$\psi_j = U_j \psi_{j-1}, \qquad \lambda_m = O\psi_m, \quad O = O^\dagger$$

O は θ_j に非依存。$\lambda_{j-1} = U_j^\dagger \lambda_j$。Forward も $U_j^\dagger$ で逆再生。

$$\frac{\partial E}{\partial \theta_j} = 2\,\mathrm{Re}\,\langle \lambda_j | \frac{\partial U_j}{\partial \theta_j} | \psi_{j-1} \rangle$$

- 非局所の逆ゲートは両状態で通信が必要。
- 基本容量は $2S(n)$。40 qubit で **32 TiB** ＋作業領域。
- 逆再生・Checkpoint/再計算・Parameter-shift を I/O 込みで比較。

対象は途中測定のないユニタリー回路。内積の frontier・Frame・Layout を整合させる。

Pauli 作用・Partner 取得・内積を共通化し、**VQE の反復と勾配評価全体**を最適化する。

17. Resource Estimator：実行前に容量と時間を見積もる

Hardware Profile ＋回路＋ Plan 候補

- RAM ピーク、ストレージ、ノード数。
- SSD/HDD read/write、MPI 送受信量。
- 全状態走査換算、Checkpoint 容量。
- 予想時間、ボトルネック、残時間。

転送費用の基本モデル

$T_{\mathrm{fetch}} \approx L + B/\beta_{\mathrm{eff}}$
L：遅延、B：バイト数、β_{eff}：実効帯域。

並行処理の重なりと競合は混合負荷計測・DAG で評価する。全 Task の費用の単純和にはしない。

取得候補の比較例（説明用の仮定値）
同じ 256 MiB Block の正しい版が取得可能、遅延を省略、処理の重なりなしと仮定。

経路	仮定	費用
Local SSD	2 GiB/s	125 ms
Remote RAM	8 GiB/s	31 ms
Log 再生	40 ms	40 ms

Log 再生は**同じ base が RAM にある**場合。base 取得・ログ転送が必要ならその費用も加算。

MPI が 1 GiB/s へ低下すると 250 ms
同じ Block でも、実効性能の変化で選ぶ経路が変わる。

見積りに**前提と誤差**を添え、実行 trace で校正する。例の数値は性能測定結果ではない。

18. 容量の基準：状態 1 個と、実行用の確保量を分ける

complex128（振幅 16 byte）、圧縮なし。以下は構成・回路を仮定した容量計算で、実機性能の測定値ではない。

状態サイズと分散した担当量

$S(n) = 16\,2^n$ byte
$s = S/N$：N node へ均等配置した担当量。
$B = 16\,2^{24} = 256$ MiB：Block 例。

- 1 qubit 増えると、状態容量は 2 倍。
- Frame ／遅延 Log は処理を減らす。
  任意の全振幅を常に圧縮できるとはしない。
- old/new、勾配 2 状態、Checkpoint、
  I/O ／ MPI buffer も別に確保する。

全状態 1 個の容量

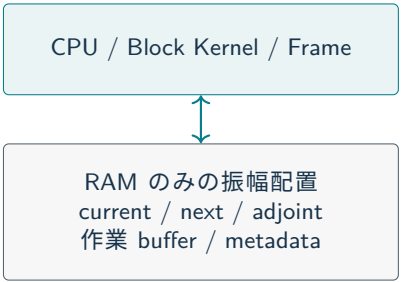
qubit	$S(n)$
30	16 GiB
32	64 GiB
36	1 TiB
40	16 TiB
44	256 TiB
50	16 PiB

GiB=2^{30}、TiB=2^{40}、PiB=2^{50} byte。
SSD の製品公称 TB と、予約する空き TiB を区別する。

容量見積りは**同時に生きる版＋蓄積する出力＋作業領域**で決まる。

19. 構成 A：単一 node・振幅を RAM だけに置く

32 qubit、$S = 64$ GiB、1 node ／ 1 rank。durable Checkpoint なし、追加 prefix・同時 Binding job なし。



状態用 SSD：0
MPI node 間通信：なし

RAM の確保例：余裕 25%を加える

$W = 16$ GiB：Block 作業領域＋ metadata。

用途	計算上の確保量	予約例
Forward	$1.25(2S + W)$ $= 180$ GiB	**256 GiB**
Adjoint	$1.25(4S + W)$ $= 340$ GiB	**512 GiB**

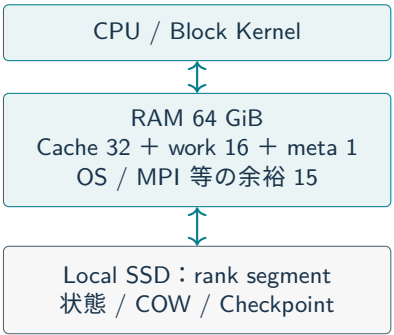
$2S$：旧／新の COW 更新。

$4S$：保護 final ＋ forward ＋ adjoint ＋次出力。
逆再生は一方ずつ更新。全中間状態を保持しない。
OS ／ MPI 内部・allocator 等の余裕は 25%に含む。

状態が 64 GiB でも RAM 64 GiB での実行を保証しない。耐久保存は別の storage が必要。

20. 構成 B：単一 node・RAM Cache + Local SSD

36 qubit、$S = 1$ TiB、1 node ／ 1 rank、$b = 24$。全状態を RAM に載せず Block 単位で処理する。



現行 append-only の SSD 確保モデル
初期 1 状態＋ Forward 8 回分の全状態 write。
Adjoint はさらに 16 回分の write を仮定。

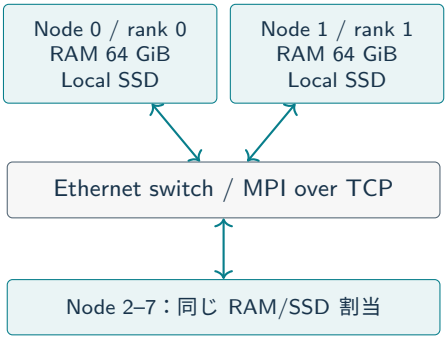
用途	余裕 25%込み	空き領域の予約例
Forward	$1.25(1+8)S$ $= 11.25$ TiB	**16 TiB**
+ Adjoint	$1.25(1+8+16)S$ $= 31.25$ TiB	**32 TiB**

更新回数は回路依存の仮定。長い回路は追加容量が必要。
payload を書かない Frame ／ virtual 更新は write 回数に入れない。

現行 Phase 3 は append-only。生存状態数個分だけで SSD 容量を保証しない。

21. 構成 C：8 node + Local SSD + Ethernet

40 qubit、$S = 16$ TiB、8 node ／ 8 rank、担当 $s = 2$ TiB/node。100 GbE を接続例として仮定（未測定）。



各 node の確保量

RAM：**64 GiB**、作業・cache 配分は構成 B と同じ。

SSD Forward：$11.25s = 22.5$ TiB
$\Rightarrow$ **32 TiB/node** を予約。

SSD + Adjoint：$31.25s = 62.5$ TiB
$\Rightarrow$ **64 TiB/node** を予約。

cluster 全体の予約例

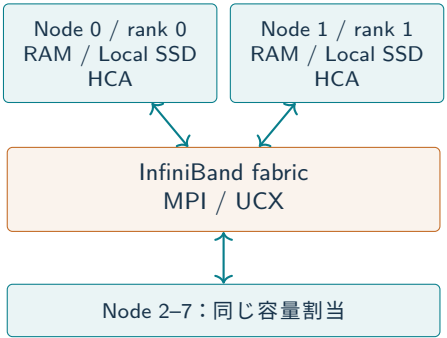
RAM：**512 GiB**
SSD：Forward **256 TiB** ／+ Adjoint **512 TiB**

source/input が remote なら MPI で取得する。
8 node の SSD を単一 node から同時 write しない。

RAM を全状態分まで増やさず容量を分散する。**通信・SSD・CPU の実効費用**は別途校正する。

22. 構成 D：8 node ＋ Local SSD ＋ InfiniBand

比較のため構成 C と同じ 40 qubit・8 node。200 Gb/s fabric は接続例の仮定で、性能測定値ではない。



確保量は Ethernet の例と同じ

RAM：**64 GiB/node**（全体 512 GiB）。
SSD：**32 TiB/node** ／＋ Adjoint **64 TiB/node**。
全体：**256 TiB** ／＋ Adjoint **512 TiB**。

RDMA 利用時の追加確認

登録・pin する通信 buffer は RAM 内で計上。
memlock 上限、UCX ／ MPI 対応、NUMA 配置、
同時 request 数を実機で確認する。

buffer の pin と追加コピーを二重計上しない。
overlap のため並列度を増やすなら RAM も再見積りする。

InfiniBand で**状態容量は減らない。**変わるのは通信の費用と、得になる実行計画。

23. 4 構成の確保例：Forward と Adjoint を分ける

容量計画の例。SSD は現行 append-only、初期 1 ＋ Forward 8 ＋ Adjoint 追加 16 状態 write、余裕 25%。

構成	node 数 qubit	状態 1 個 S	RAM / node Forward / ＋ Adjoint	SSD 空き領域 / node Forward / ＋ Adjoint
A：振幅 RAM のみ	1 / 32	64 GiB	**256 / 512 GiB**	**0 / 0**（耐久保存なし）
B：Local SSD	1 / 36	1 TiB	**64 / 64 GiB**	**16 / 32 TiB**
C：Ethernet ＋ SSD	8 / 40	16 TiB	**64 / 64 GiB**	**32 / 64 TiB**
D：InfiniBand ＋ SSD	8 / 40	16 TiB	**64 / 64 GiB**	**32 / 64 TiB**

回路・job を増やすとき
SSD は state 相当の write 回数で増える。
追加 prefix ／ checkpoint コピー／ export ／
同時 Binding の COW と作業 buffer を加算する。

生存版回収が成立する将来モデル
Forward $1.25 \times 3s$、Adjoint $1.25 \times 5s$。
ただし現行 whole-segment GC では、
この小さい容量を保証しない。

予約は**空き領域・node ごとの peak** で判定する。全体合計や NIC 帯域だけで収容可否を決めない。

24. 容量に収まる量子化学 VQE：活性空間を選ぶ

CAS=(活性電子数, 空間軌道数)。Jordan–Wigner、taper なしで $n = 2\times$ 空間軌道数。項数 M は未計測の入力仮定。

構成	量子化学の想定	qubit	状態 1 個 S	Pauli 項数 M
A：RAM	H_2O 型 CAS(8e,12o)	24	256 MiB	5,000
B：SSD	N_2 型 CAS(10e,14o)	28	4 GiB	20,000
C：Ethernet	N_2 型 CAS(10e,16o)	32	64 GiB（8 GiB/node）	50,000
D：InfiniBand	C と同じ問題・8 node	32	64 GiB（8 GiB/node）	50,000

共通 ansatz・反復条件
Hartree–Fock ＋選択した実 double excitation 16 個。
$16 \times 8 = $ **128 native Pauli rotation**、16 parameter。
100 反復、各反復で energy ＋ analytic adjoint。
full UCCSD・chemical accuracy の保証ではない。

append-only の容量確認
初期 1 ＋ Forward 128 ＋逆伝播 256 ＋ $O\psi$ 生成 1。
余裕込み **482.5 状態分／ Binding** を確保。
B：**1.885 TiB**、C/D：**3.770 TiB/node**。
前頁の SSD 予約 32 ／ 64 TiB/node に収まる。

各反復終了後に**旧 Binding scratch を撤去**して peak を抑える。100 反復の全振幅を保存しない。

25. 時間モデル：Hamiltonian の全項走査を数える

Phase 4 の条件付き計画値：並列 native kernel と logical Block 再利用を仮定。現行 Phase 3 の速度ではない。

energy ／ $O\psi$ の主な費用

$s = S/N$、M：Pauli 項数。

$$T_{\mathrm{CPU}} \approx M\,2^n/(N\rho)$$

$$T_E \approx \max(T_{\mathrm{CPU}}, 2Ms/\beta_R) + 2Ks/\beta_D + K_r s/\beta_N$$

K：upper-X mask の取得 group 数。
K_r：remote group 数、M とは別。
A：$1/0$、B：$16/0$、C/D：$163/132$。

native rotation ／ inverse ／ generator 内積、
Checkpoint・同期の費用も総時間へ加える。

計画に使う実効値の仮定

kernel / 経路	仮定値 / node
Pauli 処理 ρ	**1 億 amplitude-term/s**
RAM β_R	32 GiB/s
SSD β_D	2 GiB/s
Ethernet β_N	8 GiB/s
InfiniBand β_N	16 GiB/s

すべて未測定の仮定／目標。NIC 公称値ではない。
同時 read/write ／ peer 競合は実機で校正する。
計算＋ I/O ＋通信を加算し、overlap の加速は見込まない。

状態を運ぶ回数 K と、Pauli を評価する回数 M を分ける。M が多いと CPU が律速になる。

26. 想定処理時間：energy・勾配・100 反復を分ける

前頁の **native 並列化目標**に基づく計算値。積分生成／ SCF ／ line search 追加評価は除く。100 回の収束は保証しない。

構成 / 化学 workload	1 energy Forward 込み	1 energy ＋ Adjoint	100 反復合計
A：RAM / 24Q	約 **14 分**	約 **30 分**	約 **2.1 日**
B：SSD / 28Q	約 **15.1 時間**	約 **32.1 時間**	約 **134 日**
C：Ethernet / 32Q	約 **75.3 時間**	約 **159.2 時間**	約 **663 日**
D：InfiniBand / 32Q	約 **75.3 時間**	約 **159.0 時間**	約 **662 日**

今回の律速

Hamiltonian 5 千〜5 万 Pauli 項の内積／ action。
InfiniBand でも CPU 走査量は減らない。
M、kernel 実効値、追加評価数で時間は変わる。

現行 Python との違い

local 内積：約 **160 万 amplitude-term/s**。
表の目標 1 億は約 60 倍。
現行 runtime の量子化学実測として扱わない。

容量に収まることと、実用時間で終わることは別。計算条件・script・JSON を Markdown にも保存する。

27. 開発ロードマップ：意味の正しさから実機評価へ

設計要素は初期から型・不変条件に含め、実行 Backend と最適化を段階的に拡張する。

段階	主な実装	次へ進むための確認
1	NumPy 参照・意味モデル	8〜12 qubit で DAG / Log / Frame / Layout、任意 U・MCX・bind の数値一致。
2	単一ノード RAM ＋ SSD	Block I/O、遅延具体化、Frame・相対位相、Checkpoint 復旧を検証。
3	MPI ＋ SSD	Joint / SWAP / 取得経路の版整合。1 / 2 / 4 rank で照合。
4	期待値・勾配・反復	参照期待値・有限差分勾配と照合。prefix 再利用と追加 I/O を検証。
5	Guard・Estimator	帯域低下注入で再計画を検証。32〜36 qubit 実測から 40 qubit 級の容量・時間を推定。

比較対象を揃える
通常の Block batching / deferred-no-frame / Frame 共有 / SWAP なし・あり / 固定 Plan・適応 Plan。

記録する指標
数値誤差、総時間、I/O・MPI バイト、Cache ヒット、移行・再計画費、再開性、推定誤差。

最初の成果物：**不変条件の ADR ＋小規模参照一致＋版付き trace**。その上で改善量を測る。

根拠資料：`plan/`内の開発仕様。各フレームの TeX 内 note に参照節を記載。